

Day 6 — LangGraph, agents, multi-agent systems, and controlled autonomy

Outcome

Be able to design an agentic workflow whose state, choices, side effects, recovery, permissions, cost, and audit trail remain controlled.

1. Workflow, agent, and hybrid

Deterministic workflow:

```
code selects every next step
```

Agentic workflow:

```
model selects among bounded actions at runtime
```

Hybrid:

```
deterministic outer workflow  
→ bounded model decisions inside selected nodes
```

Use deterministic rules for known critical workflows. Use agent choice when request variability creates enough value to justify unpredictability, cost, tests, and guardrails.

Plain RAG answers with knowledge. An agent can decide to retrieve, call tools, loop, ask for approval, or escalate.

Short-term and long-term memory

- Short-term memory is current conversation/workflow context.
- Long-term memory retains selected facts, preferences, or prior cases across sessions.

Do not treat an unbounded transcript as memory. Store deliberately, retrieve only relevant items, enforce tenant/privacy/retention rules, and distinguish recalled facts from current operational truth.

2. LangGraph building blocks

State and schema

State is the explicit workflow record:

```
{
  "request_id": "...",
  "tenant_id": "...",
  "messages": [...],
  "intent": "...",
  "evidence": [...],
  "proposed_action": {...},
  "approval": "pending",
  "tool_result": None,
  "attempt_count": 0,
  "errors": []
}
```

Store raw domain state rather than display-formatted strings. Keep the schema minimal, typed, versioned, and compatible with checkpoints.

Reducers define how parallel/node updates combine. Shared mutable data without clear merge semantics creates lost updates or duplication.

Nodes

A node performs one cohesive step:

- classify;
- retrieve;
- plan;
- validate;
- request approval;
- execute tool;
- synthesize;
- escalate.

Small nodes improve testing, retry policies, metrics, and ownership. A node should not hide an entire uncontrolled application.

Edges and routing

- Fixed edge: always move to the same node.
- Conditional edge: route from state.
- Sequence, branch, loop, parallel fan-out, and approval gate.

Prefer code/business-rule routing when the rule is known. Use model classification only when variability requires it, then validate allowed routes.

Tool execution

Before:

- authenticate and authorize;
- validate schema and business rules;
- check tenant scope and approval;
- assign idempotency key;
- record intended action.

After:

- validate result;
- persist outcome;
- audit;
- decide retry, next step, or escalation.

Checkpointing and durability

Checkpoint significant state so execution can pause and resume after:

- process restart;
- human approval;
- long external wait;
- transient dependency failure.

Durable execution preserves workflow progress. It does not automatically reverse a real-world action.

A thread/workflow identifier selects the correct persisted execution history. Development may use an in-memory saver, but production recovery requires a persistent checkpointer. Resume must use the same trusted workflow identity and a compatible state/graph version.

The source notes describe three durability choices: persist on exit, persist asynchronously while the next step proceeds, or persist synchronously before advancing. They trade write latency against how much completed progress may need replay after failure. Exact names and behavior are version-sensitive, so verify them against the pinned runtime before implementation.

A resumed node may restart from its beginning rather than continue at a CPU instruction. Any side effect before an interrupt or checkpoint boundary therefore needs idempotency and a recorded outcome.

Human in the loop

Use human review for high-impact, ambiguous, or low-confidence decisions. Show the proposed action, target, inputs, evidence, and effect.

Approval can become a bottleneck; target it by risk instead of requiring it indiscriminately.

Streaming

Stream meaningful events:

```
classified
retrieving
evidence_found
approval_required
tool_started
tool_completed
final_answer
```

Streaming improves experience but complicates cancellation, partial errors, moderation, and rollout draining.

3. Example controlled state machine

```
RECEIVED
→ CLASSIFIED
  ├── policy question → RETRIEVE
  ├── live status → READ_TOOL
  ├── proposed change → PLAN
  └── unsafe/unknown → ESCALATE

PLAN
→ VALIDATE
  ├── forbidden → FAILED_POLICY
  ├── approval needed → WAITING_APPROVAL
  └── allowed → EXECUTING

WAITING_APPROVAL
  ├── approved → EXECUTING
  └── rejected → CANCELLED

EXECUTING
→ OBSERVATION_RECORDED
  ├── more bounded work → NEXT_STEP
  └── done → COMPLETED
```

Persist before external side effects and after learning the outcome.

4. Reliability and recovery

Bound every loop

Use:

- maximum steps;
- maximum retries per node/tool;
- workflow deadline;
- token/cost budget;
- repeated-state detection;
- explicit terminal and escalation states.

Retry rules

Retry transient, idempotent operations with backoff and limits.

Do not blindly retry:

- invalid inputs;
- policy denial;
- non-idempotent writes with unknown outcome;
- deterministic business rejection.

Replay versus rollback

Replaying from a checkpoint reproduces workflow execution. It does not undo an email, reservation cancellation, payment, or deployment.

For side effects use:

- idempotency;
- reconciliation;
- compensating business action where valid, often coordinated as part of a saga;
- human/operator escalation.

Graph changes

Persisted states may outlive code versions. Version graph/state schemas and define compatibility or migration. Do not assume old checkpoints safely resume under new semantics.

Fallbacks

- retrieval failure → controlled answer/escalation;
- reranker failure → first-stage rank if acceptable;
- model failure → approved fallback;
- tool timeout → reconcile outcome before retry;
- approval timeout → expire/escalate;
- checkpoint failure → stop before unsafe side effect.

5. Agent safety

- Allowlisted tools.
- Narrow typed schemas.
- Separate read and write capabilities.
- Server-side authorization.
- Tenant-scoped credentials.
- Egress restrictions.
- Approval for consequential actions.
- Step/time/token/cost limits.
- Result-size limits.
- Output sanitization.
- Immutable audit.

Prompt instructions are not authorization.

| The model may propose an action; policy authorizes it.

Retrieved/tool content is untrusted and cannot grant permissions or introduce new tools.

6. Multi-agent roles and patterns

Possible roles:

- Planner: decomposes work.
- Researcher/retrieval agent: gathers evidence.
- Router: selects specialist.
- Critic/reviewer: checks quality/risk.
- Executor: performs an approved action.
- Escalation agent: prepares a human package.

Patterns:

Pattern	Flow
Sequential	Agent A → B → C
Manager-worker	Manager delegates and combines
Router-specialist	Router selects domain expert
Planner-executor-verifier	Plan, act, verify
Debate/critic	Candidate plus critique/revision
Human review	Agents prepare; human decides

When useful

- distinct specialist tools/knowledge;
- complex tasks with review;
- parallel independent research;
- organizational role boundaries;
- reusable specialist capabilities.

When overkill

- one retrieval and generation call;
- fixed linear steps;
- one agent with a few tools is sufficient;
- latency/cost/audit requirements dominate;
- roles share the same context and add no real specialization.

More agents add:

- model calls;
- state and message coordination;
- failure combinations;
- latency and cost;
- conflict resolution;
- governance and audit surface.

Shared state needs explicit ownership and merge rules; otherwise agents overwrite, duplicate, or trust stale observations.

7. Framework positioning

Item	Source-note mental model	Core concepts / boundary
LangGraph	Explicit workflow graph/state machine	State, nodes, edges, conditional routes, checkpoints, approval, recovery.
CrewAI	Role-based AI team plus flow	Agent, task, crew, process, tool, and stateful flow; useful for collaborative role/task composition.

Semantic Kernel	Enterprise AI application integration SDK	Kernel, plugins, functions, memory, filters, function calling, and process/agent concepts.
AutoGen	Multi-agent conversational patterns	Agent conversations, teams, and human-agent collaboration.
Microsoft Agent Framework	Newer production-focused Microsoft framework in the source notes	Combines AutoGen-style agent abstractions with Semantic Kernel-style state, type safety, middleware, telemetry, filters, and graph workflows.
ADK	Agent development framework in the source notes	Build, debug, deploy, and orchestrate agents.
A2A	Agent-to-agent protocol	Lets agents built by different teams/frameworks communicate; it does not build the agent.
n8n/low-code	Visual deterministic integration workflow	Useful when explicit application/integration steps matter more than agent autonomy.
MCP	Host-to-capability connectivity protocol	External tools, resources, and prompts; not agent coordination.

Frameworks build/orchestrate agents; protocols connect boundaries. In the source-note mapping, ADK can build an agent while A2A lets that agent communicate with other agents. Semantic Kernel and AutoGen concepts feed the newer Microsoft Agent Framework positioning. MCP can still connect any of these systems to external capabilities.

Tool/function/plugin/skill terminology varies:

```

tool or function = one callable capability
plugin = related functions exposed together
skill = reusable capability or instruction pattern

```

Choose based on workflow control, integration needs, team operations, evaluation, and governance—not labels.

8. Testing and evaluation

Unit

- each node;
- routing functions;
- state reducers;
- tool input/output translation;

- policy and approval rules.

Scenario

- happy path;
- low evidence;
- malformed model output;
- tool denial;
- timeout and retry;
- duplicate callback;
- human approval/rejection/expiry;
- loop limit;
- resume from checkpoint;
- state compatibility.

Metrics

Workflow:

- completion/abandonment;
- end-to-end duration;
- active and stuck workflows;
- checkpoint/recovery failures.

Node/routing:

- latency/error/retry;
- route distribution and accuracy;
- model calls/tokens.

Tools:

- selection accuracy;
- success/error/unknown outcome;
- authorization denial;
- idempotency conflicts.

Human:

- approval/rejection;
- wait time;
- override rate.

Business:

- successful task completion;
- cost per completed task;
- escalations and user feedback.

9. Production example: IT support agent

```
user request
→ classify
→ retrieve runbook
→ check live incident status with read tool
→ assemble evidence
→ decide:
    answer directly
    propose ticket/update
    request approval
    escalate
→ execute narrow authorized tool
→ validate result
→ final answer with evidence and action ID
```

State contains the original request, identity scope, retrieved runbook, live status, proposal, approval, tool outcome, and audit identifiers.

The model never receives a broad admin token and never authorizes its own action.

10. Trade-offs

- Free-form agent: flexible, difficult to predict.
- Explicit graph: controllable and testable, requires workflow design.
- Durable state: resumable, adds storage/versioning/operations.
- Human approval: safer, adds delay and workload.
- Multi-agent: specialized/parallel, adds coordination and cost.
- Coarse nodes: fewer transitions, less visibility.
- Fine nodes: better control/telemetry, more workflow sprawl.

Project-grounded example: controlled autonomy in BenchOps Copilot

Project scenario. DPDK BenchOps Copilot had to answer variable natural-language questions and assist with benchmark plans, but incorrect commands or disruptive BIOS guidance were unacceptable. The documented LangGraph/LangChain flow identified intent, retrieved benchmark context, filtered by workload/platform/source metadata, called deterministic tools when needed, verified evidence support, and returned cited structured answers.

How the concepts apply. This is a hybrid agent:

```
model-assisted:
  interpret the request
  synthesize retrieved evidence
  explain metrics or regression context

deterministic:
  query runs and metadata
  fetch logs/artifacts
  compare runs
  validate plans
  build commands from allowlisted templates
  parse results
  gate BIOS/reboot-affecting actions
```

The model could choose or propose a bounded capability, but the tool implementation and safety boundary determined what was actually possible. A verification step checked that the response was supported by retrieved context and tool results.

Design decisions and trade-offs.

- **Explicit graph over a free-form agent:** more workflow design and node-level testing, but clearer control over tool use and evidence verification.
- **Deterministic tools over generated shell commands:** narrower behavior, but reproducible commands and lower operational risk.
- **Human control for BIOS/reboot operations:** slower completion, but a proportionate control for disruptive actions.
- **Verification before response:** additional latency, but a direct mitigation for unsupported benchmark guidance.

Outcome. The project reports safer operational workflows, grounded cited answers, auditable tool calls, and higher release confidence through golden-set CI evaluation. It does not provide a measured agent-completion rate, average step count, or approval wait time.

Senior/Staff interview framing.

- **Senior:** draw the node/edge flow for a tuning question and a command-plan request. Identify the tool schemas, failure branches, loop/attempt bounds you would require, and the evidence needed before answering.
- **Staff:** explain how you allocated autonomy by risk. Tie graph structure, tool ownership, approval, audit, evaluation, and deployment rollback into one governance model, and state what evidence would justify giving the model more or less freedom.

Evidence boundary and topic gap. The project narrative does not document checkpoint persistence, replay behavior, long-term memory, or a multi-agent implementation. Do not turn the retrieval, tool, and verifier stages into fictional “agents.” If asked about multi-agent systems, use this project to explain why a controlled single workflow was sufficient and describe multi-agent adoption only as a **hypothetical** option requiring real specialization or parallel value.

11. Interview questions

1. Workflow versus agent versus hybrid?
2. When is a chain insufficient?

3. What belongs in state?
4. Why keep nodes small?
5. How do fixed and conditional edges differ?
6. What does checkpointing provide?
7. Why is replay not business rollback?
8. How do you bound an agent loop?
9. How do you retry a timed-out write tool?
- 10
 - . Where should authorization occur?
- 11
 - . When is multi-agent useful, and when is it overkill?
- 12
 - . What metrics prove an agent is useful, safe, and affordable?
- 13
 - . Why must workflow identity and graph/state version accompany a checkpoint?
- 14
 - . How do stronger durability settings trade latency against recovery work?

12. Exit checklist

- [] Draw an explicit state schema and graph.
- [] Separate deterministic and model-driven decisions.
- [] Add checkpoint, approval, limits, idempotency, and audit.
- [] Explain recovery and unknown external outcomes.
- [] Choose single versus multi-agent patterns.
- [] Design unit, scenario, failure, and evaluation tests.
- [] Explain framework roles without confusing MCP and orchestration.
- [] Explain persistent checkpoint identity, resume semantics, and side-effect idempotency.

Source notes

- [LangGraph and Agentic AI](#)
- [Multi-Agent AI Overview](#)
- [Enterprise GenAI Solution Design](#)
- [Trees, Graphs, and DP](#)
- [LangChain End to End](#)
- [LangGraph End to End](#)
- [MCP End to End](#)
- [Vanilla RAG and Frameworks](#)
- [Capstone Revision Day 2](#)
- [Capstone Revision Day 3](#)
- [DPDK Automation for Network Packet Processing](#)
- [DPDK BenchOps Copilot](#)